

# Comparing Rule-Based, Game-Theoretic, and Reinforcement-Learned Response Selection for Residential IoT Defense: A Controlled Mininet Evaluation with Disclosed Self-Correction

Author Name\*

\*Affiliation

Email: author@example.com

**Abstract**—Automated response selection for residential IoT networks is typically evaluated either as a detection problem (is the traffic malicious?) or through anecdotal demonstration, rarely as a controlled comparison of how the response itself is chosen. We present a registry-driven defense system for a Mininet-simulated residential IoT network that compares three independent response-selection policies – a rule-based classifier, a Stackelberg game-theoretic solver, and a PPO-trained reinforcement-learning agent – against two naive baselines and an external signature-based IDS (Suricata), on real captured traffic across 17 threat conditions (16 attack types plus benign). Our evaluation harness runs 255 real Mininet trials per reported round (1,275 recorded outcomes), reporting Wilson-score confidence intervals rather than point estimates, and includes ablations on the Stackelberg payoff table, PPO reward shaping, training-seed robustness, and decision-boundary policy divergence, backed by a formal paired significance test (McNemar’s exact test) rather than agreement-rate percentages alone. What distinguishes this evaluation from a typical system paper is not its headline accuracy but the process that produced it. Re-running the harness at three times an initial sample size first revealed that several headline numbers had been optimistic at low  $N$  (aggregate detection accuracy fell from 90.0% to 73.7%); rather than accept that as a ceiling, we investigated each underperforming condition as a live, reproducible bug, found and fixed seven real defects (a capture-race condition, a traffic-pacing error, a silently-wrong protocol default, two Suricata-ruleset gaps, a reward-function asymmetry, and a packet-buffering bug), and re-ran the full harness under the fixed code: aggregate detection accuracy reached 99.6% [97.8, 99.9], with 16 of 17 conditions scoring a clean 100%. One further attempted fix – quadrupling PPO’s training budget to eliminate a residual seed-dependent failure – was tried, measured on a full ten-seed sweep, found not to generalize from the single seed it was validated on, and reverted; we report this as a documented dead end, not a success. We conclude that the three response-selection policies are behaviorally indistinguishable on in-distribution traffic (McNemar’s exact test,  $p = 1.0$ ) but diverge significantly near decision boundaries ( $p < 1.2 \times 10^{-159}$ ), that our system now materially outperforms naive baselines by a wide, formally-tested margin, and that every result in this paper is scoped to a single controlled VM and Mininet topology – external validity remains an explicitly open problem, and internal validity was

earned through disclosed, repeated self-correction rather than assumed.

**Index Terms**—IoT security, intrusion response, Stackelberg games, reinforcement learning, Mininet, reproducibility

## I. Introduction

Consumer and residential IoT devices are widely deployed with weak or absent built-in security, and network-level defense – detecting an attack and then choosing what to do about it – is increasingly delegated to automated systems. The detection side of this problem is well studied: signature-based tools such as Snort and Suricata, and a large body of machine-learning intrusion-detection literature, address the question of whether observed traffic is malicious. The response-selection side – given a detected threat, which of several possible actions (alert, isolate, deceive, rate-limit, quarantine, ...) should actually be taken – receives comparatively little controlled, comparative evaluation. Individual mechanisms (game-theoretic defense, reinforcement-learned defense, moving-target defense) are each well represented in the literature, but rarely compared against each other, against simple baselines, and against an external detector, on the same captured traffic, in the same controlled environment.

This paper makes four contributions:

- 1) A registry-driven defense architecture for a simulated residential IoT network (Section III) in which a single declarative record per attack type drives traffic generation, detection, Stackelberg payoff lookup, PPO observation encoding, and dataset generation, so that adding a new attack type or defense action does not require touching each subsystem by hand.
- 2) A repeatable benchmark harness (Section IV) that runs three independent response-selection policies – rule-based, Stackelberg game-theoretic, and PPO reinforcement-learned – alongside two naive baselines and an external, real signature-based IDS, against the same captured Mininet traffic per trial,

with every chosen action actually executed and independently verified rather than assumed successful.

- 3) A set of ablations and robustness checks (Section V) largely absent from comparable systems work: Wilson-score confidence intervals on every reported rate, a payoff-table sensitivity ablation, a reward-shaping ablation, a ten-seed PPO training-robustness sweep, a direct test of policy divergence at synthetic decision boundaries, and a formal paired significance test (McNemar’s exact test) confirming that divergence rather than resting on agreement-rate percentages alone.
- 4) A transparent account of three self-corrections (Section VI): a threefold sample-size increase that first revealed several headline numbers had been optimistic, followed by a systematic bug-hunt that found and fixed six real defects and recovered a 99.6% aggregate accuracy; an adaptive-attacker scale-up that discovered a real bug in our own evaluation code and, once fixed, failed to reproduce an earlier finding; and a training-budget fix that resolved one isolated failure but did not generalize on full re-test, and was reverted. All three are reported as corrections, including the one that failed after already being committed, rather than suppressed or quietly kept.

We deliberately scope every claim in this paper to the environment it was measured in: a single Mininet topology on one resource-constrained virtual machine (Section VII). We do not claim external validity, and we treat the absence of that claim as a feature of a rigorous internal evaluation, not an oversight to be hidden.

## II. Related Work

No published study was found comparing rule-based, Stackelberg game-theoretic, and reinforcement-learned response selection for IoT defense on a shared dataset; the closest work in the literature compares detection methods alone, or evaluates a single game-theoretic or learned defense mechanism on its own metrics and environment. We situate our design choices against this literature as context, not as a claimed head-to-head benchmark.

Signature-based detection. An experimental comparison of Suricata and Snort found Suricata generally ahead on detection accuracy, scalability, and resource efficiency (e.g. 100% vs. 85.7%/66.7% on two DNS-tunneling variants) [1], motivating our choice of Suricata as the external detection baseline (Section IV-C). Prior work evaluating signature-based tools against IoT botnet datasets (ISOT, IoT-23, Bot-IoT) documents the same structural limitation we independently observe: signature/threshold detectors miss traffic that does not match a known pattern [2].

Game-theoretic and deception-based defense. A stochastic-game honeypot moving-target-defense study reports substantial real engagement gains from deception over a static defense (attacker packet interactions

up 920 vs. 42; scanner engagement extended 14.6 $\times$ ) [3] – a different environment and metric than our controlled, single-attacker Mininet trials, but directionally consistent with our own DECOY action’s measured value (Section V-B). A 2025 paper models collaborative IoT packet-sampling against DDoS as a Stackelberg game and derives a sampling-rate lower bound that deters an attacker [4] – the same game-theoretic framing we use, but for a continuous sampling-rate decision rather than a choice among discrete response actions, and evaluated analytically rather than against captured traffic.

Reinforcement learning for defense. Recent surveys document deep RL as an active, promising direction for IoT intrusion detection while flagging reproducibility and dataset-realism concerns across the field [5] – concerns we address directly by training and evaluating our PPO policy against real, verified Mininet outcomes and by reporting a seed-robustness sweep (Section V-E) rather than a single trained checkpoint. A 2025 paper evaluates an RL-tuned moving-target-defense mutation strategy for edge IoT against DDoS specifically on attack success rate, defense latency, and CPU overhead [6] – the same category of cost-conscious framing our own cost/overhead analysis uses (Section V-D), for mutating network configuration rather than selecting among discrete response actions.

None of the above papers shares our dataset, environment, or exact decision space, which is precisely why our results are reported as an internally-valid comparison between the arms tested here, not translated into a claimed standing against any of them.

## III. System Architecture

### A. Registry-driven attack and response model

The system is organized around a single declarative registry (AttackScenario) with one record per attack type, holding its traffic generator, rule-based detector, registered “preferred” response action, and the confidence/score thresholds that action requires. Every dependent subsystem – the rule-based decision policy, the Stackelberg payoff lookup, PPO’s one-hot scenario encoding, and dataset generation for the machine-learning baseline – derives its behavior from this registry rather than encoding per-attack logic separately in each place. This was a deliberate response to an early failure mode in the project’s own development: adding one new attack type by hand across six independently-maintained files produced four distinct, reproduced bugs (a malformed traffic-generation script that hung indefinitely, an insufficient process-termination grace window, a privileged-port binding failure, and output-buffering interference with the network simulator’s own control channel). The registry refactor eliminates that failure class by construction for every attack added afterward.

TABLE I  
Registered threat conditions and preferred responses

| Condition                  | Signature axis                       | Preferred            |
|----------------------------|--------------------------------------|----------------------|
| normal                     | – (benign baseline)                  | ALLOW                |
| reconnaissance port scan   | many ports, moderate rate            | DECOY                |
| DoS flood                  | one port, very high rate             | ISOLATE              |
| brute force                | one port, sustained attempts         | BLOCK<br>_SOURCE     |
| data exfiltration          | reversed direction, large payload    | ISOLATE              |
| exploit payload injection  | oversized single payload             | DECOY                |
| TCP SYN flood              | refused connects, one port           | ISOLATE              |
| ICMP ping flood            | high-volume padded ICMP              | THROTTLE             |
| slow-loris exhaustion      | many held-open connections           | THROTTLE             |
| DNS amplification          | oversized inbound UDP                | BANDWIDTH<br>_CAP    |
| DNS tunneling exfiltration | reversed, many small UDP             | DECOY                |
| MQTT message flood         | many completed TCP conns.            | THROTTLE             |
| firmware tampering         | reversed, periodic oversized push    | QUARANTINE           |
| buffer overflow probe      | sustained oversized payload, 1 conn. | RESET<br>_SESSIONS   |
| credential replay          | repeated uniform small UDP           | BLOCK<br>_SOURCE     |
| rogue config beacon        | reversed, low-confidence pattern     | FORENSIC<br>_CAPTURE |
| C2 beaconing               | periodic inter-arrival timing        | ISOLATE              |

## B. Threat model and attack catalog

Table I summarizes the 16 registered attack types plus the benign (normal) condition, each with a distinct traffic signature and a registered preferred response. Signatures span packet rate, port diversity, payload size, traffic direction (several attacks, e.g. data\_exfiltration and dns\_tunneling\_exfiltration, reverse the usual device-is-target direction), and, for c2\_beaconing, inter-packet timing regularity – the only signature in the registry that is not rate-, volume-, or size-based. Every detector’s numeric threshold window was placed in a verified, unclaimed gap relative to every other detector’s window, confirmed both by a parametrized test sweeping every attack’s synthetic signature through the complete detector set and by repeated live Mininet runs under real timing variance.

## C. Defense actions

Ten actions are available to all three policies, all real, verified mechanisms executed inside the Mininet network rather than simulated effects: ALLOW and ALERT (no enforcement); ISOLATE (interface down, verified by polling link state); DECOY (a real TCP banner service with NAT redirection, verified by a real redirected connection); THROTTLE (a protocol-aware iptables hashlimit rate cap, verified against real traffic – an unthrottled attacker completed 56/56 connection attempts in 3s, the same traffic under a 2/s cap completed 7); BLOCK\_SOURCE (a source-scoped iptables DROP

rule, verified by polling the installed rule); QUARANTINE (default-deny with an allowlist for known-legitimate hosts); RESET\_SESSIONS (real kernel socket destruction via ss -K); FORENSIC\_CAPTURE (preserves the detection pipeline’s own already-captured evidence); and BANDWIDTH\_CAP (a tc tbfb egress cap on the attacker’s interface, verified at the receiver: 300/300 packets delivered uncapped vs. 20/300 under the cap).

## D. Response-selection policies

Rule-based. A deterministic mapping from the classified attack type and its score/confidence to the registry’s preferred action, with dedicated handling for the benign and low-confidence cases.

Stackelberg. A leader-follower game solved per detected threat type, with a full, independently-reasoned payoff table (attacker and defender utility for every action, for every threat type, in config/policies.yaml) rather than a derived or auto-generated one. The solver’s own selection was verified to agree with the registry’s declared preferred action for all 17 conditions.

PPO. A proximal-policy-optimization agent (Stable-Baselines3) over a ten-action discrete space matching DefenseAction’s declaration order, trained against a deterministic normalized security-context observation. The deployed recipe is net\_arch=[64,64], ent\_coef=0.01, n\_steps=batch\_size=170, learning rate 0.001, 25,500 timesteps. An initial net\_arch=[32,32] network reliably mis-routed the benign scenario to DECOY regardless of training length; doubling network capacity resolved this at the project’s development seed. Section V-E reports a formal, multi-seed robustness check of this recipe rather than relying on that single seed’s success.

## IV. Evaluation Methodology

### A. Harness design

Our benchmark harness runs  $N$  real Mininet trials for every registered condition. Each trial (i) generates real traffic for that condition (the same generators the live system uses) and captures it, preserving the raw packet capture for later offline analysis; (ii) classifies the capture with the attack-type-agnostic rule-based detector; (iii) evaluates five arms against the identical resulting security context – the rule-based, Stackelberg, and PPO policies, plus NaiveBlockAllBaseline (isolate any detected threat, the common “one fixed response to every alert” design) and AlwaysAllowBaseline (never intervene, the floor); and (iv) executes and independently verifies each distinct action chosen across the five arms exactly once, for real. One trial produces every arm’s result; no arm requires a separate live run.

### B. Metrics

Detection accuracy: did the shared detector correctly classify the captured traffic (one result per trial, shared across arms, reported as a sanity check rather than a

TABLE II

Five-arm comparison,  $N = 15$  trials/condition ( $n = 255$  trials), post-fix

| Arm                | Detection acc.   | Verified resp.   | Matches pref.    |
|--------------------|------------------|------------------|------------------|
| Rule-based (ours)  | 99.6[97.8, 99.9] | 99.6[97.8, 99.9] | 99.6[97.7, 99.9] |
| Stackelberg (ours) | 99.6[97.8, 99.9] | 99.6[97.8, 99.9] | 99.6[97.7, 99.9] |
| PPO (ours)         | 99.6[97.8, 99.9] | 99.6[97.8, 99.9] | 99.6[97.7, 99.9] |
| NaiveBlockAll      | 99.6[97.8, 99.9] | 29.4[24.2, 35.3] | 25.0[19.9, 30.8] |
| AlwaysAllow        | 99.6[97.8, 99.9] | 5.9[3.6, 9.5]    | 0.0[0.0, 1.6]    |

point of comparison). Matches preferred action: of the attack trials, how often an arm’s chosen action was the registry’s declared preferred action. Verified response rate: of all trials, how often an arm both chose the preferred action and that action’s real effect was independently confirmed – our headline metric, zero whenever the wrong action was chosen, only crediting a checked success. Every reported rate carries a 95% Wilson-score confidence interval, computed directly rather than via the normal approximation (which misbehaves at small  $n$  or extreme proportions).

#### C. External detection baseline

A second, independent arm runs offline Suricata analysis of the same captured packet captures, against two rulesets: ET-Open (real-world community threat signatures) and a small lab-tailored ruleset encoding the same shape knowledge our own rule-based detectors use, so Suricata is judged on a fair signature-matching version of that knowledge too, not only on a strawman of unrelated real-world malware signatures. Suricata is compared on detection axis only (accuracy, true-/false-positive rate against ground truth); it identifies traffic but does not select or execute a containment action.

#### D. Scale

This paper reports the harness at `trials_per_condition = 15` across the full 17-condition registry: 255 real Mininet trials, 1,275 recorded outcomes, on 2026-09-25 ( $\approx 2\text{h}21\text{m}$  wall-clock, run after all seven fixes in Section VI landed). Two earlier runs at this same scale – one at  $N = 5$ , one at  $N = 15$  but against the pre-fix code (Section VI-A) – are discussed explicitly because their headline numbers did not hold, first at the larger sample and then again once the underlying code was corrected.

### V. Results

#### A. Main comparison

Table II reports the five-arm comparison after all seven bugs in Section VI were fixed and the harness was re-run in full. The three real policies are statistically indistinguishable from each other on this dataset (confirmed formally in Section V-H) and substantially outperform both baselines.

All values are percentages with 95% Wilson CIs. Sixteen of the seventeen conditions score a clean 15/15; the one

TABLE III

External detection comparison, offline analysis of 255 pcaps, post-fix

| Detector             | Accuracy         | TPR   | FPR   |
|----------------------|------------------|-------|-------|
| Suricata + ET-Open   | 43.9%            | 41.3% | 13.3% |
| Suricata + lab rules | 94.1%            | 93.8% | 0.0%  |
| Ours (rule-based)    | 99.6[97.8, 99.9] | –     | 0.0%  |

exception, `credential_replay` (93.3%), has a completely empty, unreadable pcap on its one miss – a pre-existing, disclosed capture-flakiness class distinct from the capture-race bug fixed in Section VI. Our system’s 99.6% verified-response rate is  $3.4\times$  NaiveBlockAllBaseline’s 29.4% and  $16.9\times$  AlwaysAllowBaseline’s 5.9% – a wider margin than any earlier-scale estimate, because our own numbers improved dramatically while the baselines, unaffected by any of the seven fixes, stayed roughly where they were. Mean detection latency (31,287ms) is dominated by real packet capture, not decision-making (Section V-D).

#### B. External detection comparison

Table III compares our detector against Suricata under both rulesets, after fixing `dos_flood`’s non-firing rule and writing real signatures for the eleven attacks the lab ruleset previously lacked (Section VI).

The lab-tailored ruleset now performs close to our own detector – real, independent, external validation of the fixed signatures. ET-Open’s own numbers moved too (previously 67.8% accuracy) despite nothing in this pass touching its ruleset: a real, disclosed side effect of several traffic generators changing shape this session (Section VI), not a new defect – ET-Open’s community signatures were never tuned for this lab’s synthetic traffic in the first place.

#### C. Supplementary ML confirmation layer

Separately from the three response-selection policies, a Random Forest classifier serves as a confirmation step for the one signature with the fuzziest rule-based boundary (`reconnaissance_port_scan`); it is not itself one of the compared policies. Its own training dataset was, until this pass, limited to the original 6-class registry, predating the expansion to 17 conditions. We generated a fresh, real 17-class dataset (255 Mininet runs, 0 failures, 260 rows balanced at 15/class) and retrained it: on a held-out test split (40 rows), the Random Forest reaches 97.5% accuracy (precision 91.2%, recall 92.2%) against a same-split rule-based baseline of 90.0% – a real, measured 7.5-point gap on the full registry, superseding an earlier 6-class figure that predated the ten-attack expansion. Two classes showed 0 precision/recall on this specific split at only 2–3 held-out rows each; reported as small-sample noise, not a claimed per-class weakness.

#### D. Cost of comparing three policies

Timing 300 real, direct calls per policy (isolated from Mininet and packet capture) gives mean decision la-

TABLE IV  
Stackelberg payoff-table sensitivity (2,550 solves)

| Noise level | Match rate (95% CI) |
|-------------|---------------------|
| $\pm 5\%$   | 96.1[94.0, 97.4]    |
| $\pm 10\%$  | 88.0[85.1, 90.5]    |
| $\pm 20\%$  | 78.8[75.3, 82.0]    |
| $\pm 35\%$  | 64.5[60.5, 68.4]    |
| $\pm 50\%$  | 61.0[56.8, 65.0]    |

tencies of 0.054 ms (rule-based), 0.464 ms (Stackelberg), and 1.136 ms (PPO). Running all three sequentially, as our comparison architecture does, costs 1.654 ms total; the marginal overhead of comparing three policies versus deploying only the slowest one is 0.518 ms – approximately 0.0017% of the harness’s own 31.3 s mean real detection latency. Comparing multiple policies is not this system’s bottleneck.

#### E. PPO training seed-robustness

A single trained checkpoint is a weak basis for a robustness claim. We trained 10 independent models at the deployed recipe, varying only the random seed (1, 2, 3, 7, 13, 17, 42, 55, 88, 99), and checked full convergence to every registered preferred action. An initial sweep found 8/10 fully converge (Wilson 95% CI [49.0, 94.3]), both failures the same class of mistake – a false-positive-style error on benign traffic (one seed chose THROTTLE, one chose DECOY, instead of ALLOW). Tracing this to a real reward-function asymmetry (only ISOLATE-on-normal carried an extra disruption penalty, giving the optimizer less pressure to avoid the other two actions specifically) and fixing it raised the deployed recipe’s own rate to **9/10** (Wilson 95% CI [59.6, 98.2]) – the figure Table II was trained under. A further attempt to eliminate the one remaining failure did not hold up under full re-testing; Section VI-C reports it as a documented dead end. The wide CI is reported honestly rather than rounded into overconfidence: this supports “most seeds converge fully; one specific instability mode recurs in roughly one run in ten,” not a tighter claim.

#### F. Ablations

Stackelberg payoff sensitivity. We perturbed every value in the deployed payoff table by random noise proportional to its own magnitude ( $\pm 5\%$  to  $\pm 50\%$ , 30 trials/level, 2,550 solves total) and measured the fraction of solves still matching the registry’s declared action. Table IV shows a monotonic, graceful degradation: small realistic tuning error barely moves the outcome (the qualitative payoff structure is doing most of the work), while at  $\pm 50\%$  noise over a third of solves diverge, confirming the specific values still carry real information.

PPO reward shaping. We trained the real environment under two reward configurations, holding the recipe and a five-seed sweep fixed: the deployed shaped reward

(different magnitudes for different outcome severities, e.g. letting an attack through unopposed is penalized twice as hard as choosing a merely suboptimal defensive response) against a flat reward collapsing every correct outcome to the same reward and every incorrect outcome to the same penalty. Both configurations reached full convergence on 5/5 seeds (Wilson CI [56.6, 100.0] for both). We report this honestly as inconclusive at this sample size, not as evidence that shaping is unnecessary: both intervals are wide and fully overlapping, and the more likely places shaping would matter – sample efficiency, or a longer-horizon task than our single-decision environment – were not measured here.

#### G. Decision divergence near boundaries

Because real captured traffic in our lab classifies far from any threshold every time, the three policies choosing identical actions on Table II is closer to a consistency check than a comparison. We separately tested all three policy classes directly against synthetic security contexts: (i) threat-score/confidence perturbed around each attack’s own registered decision threshold ( $\pm 0.03$  to  $\pm 0.15$ , 784 contexts across the full registry), and (ii) entirely unregistered “novel” attack types no policy was tuned for (12 contexts). Near decision boundaries, rule-based diverges from both Stackelberg and PPO on roughly two-thirds of ambiguous contexts (all-three agreement 32.5%), while Stackelberg and PPO track each other closely (100% mutual agreement) even off the training distribution – a positive finding that the learned policy generalizes toward the same strategic reasoning the game solver encodes explicitly. On entirely unregistered attack types, rule-based and Stackelberg both degrade to a safe ALERT default every time; PPO instead selects ISOLATE regardless of the actual threat score or confidence supplied – a genuine, disclosed limitation: PPO has no mechanism for recognizing “I have never seen anything like this,” unlike the other two policies’ explicit fallback.

#### H. Formal significance testing: McNemar’s exact test

Every comparison above reports agreement or verified-response percentages – real, but not a formal test of whether an observed difference could plausibly be chance. Because every arm is scored against the identical trial or synthetic context, this is a paired design, and McNemar’s test is the appropriate significance test for it: it conditions on the trials where two policies actually disagree, since a trial both get right (or both get wrong) carries no information about which is better. We implement the exact binomial form rather than the more commonly cited chi-square approximation, which is unreliable at the small discordant-pair counts a paired, mostly-agreeing comparison like this one produces, and verify our implementation against a hand-computed reference before applying it to real data.

On the 255 real harness trials (Table II), all three pairwise comparisons show zero discordant pairs ( $p = 1.0$ ) – the rigorous version of a claim this paper has otherwise only supported with matching percentages: on real captured traffic, there is no statistical evidence whatsoever of a performance difference between any pair of policies, because the three never once chose differently across these 255 trials. On the 784-context synthetic decision-boundary dataset (Section V-G), where real disagreement exists, the same test tells a sharply different story: both Stackelberg and PPO significantly outperform rule-based (529/784 discordant pairs, 100% in their favor, exact  $p < 1.2 \times 10^{-159}$ ), while remaining statistically indistinguishable from each other ( $p = 1.0$ , zero discordant pairs) – formal confirmation, not just an observed agreement rate, that the learned policy converges toward the same strategic reasoning the game-theoretic solver encodes explicitly.

#### I. Detector robustness (evasion margin)

Perturbing every numeric feature in each attack’s own signature by  $-30\%$  to  $+30\%$  and re-classifying through the complete detector set measures brittleness versus real margin. Results are uneven, not uniformly reassuring: robust classification rates range from 28.6% (dns\_amplification, firmware\_tampering, buffer\_overflow – a crowded region of the numeric feature space) to 100% (dos, brute\_force, exfiltration). We are explicit that this measures measurement-noise tolerance, not adversarial evasion resistance: the perturbation is symmetric, random, and non-adaptive, and does not search for the nearest evasive point the way a real, threshold-aware adversary would (Section VII).

### VI. Discussion: Self-Correction as Methodology

We report three findings in this section specifically because our first attempt at strengthening each one did not simply add more of the same evidence – it changed what we could honestly claim, in one case for the better once the underlying cause was fixed, and in another not at all.

#### A. Sample size revealed optimism – and investigation fixed it

An earlier harness run at  $N = 5$  trials/condition across 16 conditions reported aggregate detection accuracy of 90.0% and verified-response rate of 85.0%. Re-running at  $N = 15$  across the full 17-condition registry revised both down substantially, to 73.7% and 68.2%. Several individual conditions that read as a clean 100% at  $N = 5$  settled meaningfully lower at  $N = 15$  – exactly the outcome a single unlucky trial swinging a small-sample percentage by a full 20 points predicts. Two conditions illustrated this most sharply: tcp\_syn\_flood fell to 20.0% detection accuracy, and icmp\_ping\_flood to a reproducible 0% verified-response rate (CI [0.0, 20.4], now excluding anything above roughly one-fifth).

Rather than accept 73.7% as this system’s ceiling, we treated every underperforming condition as a live, reproducible bug and investigated each with a direct repro rather than a post-hoc explanation. Six real, distinct defects were found and fixed: (1) a capture-race condition in which non-unique capture filenames let a not-fully-terminated tcpdump process from one round silently corrupt the next round’s capture on any persistent-network module; (2) tcp\_syn\_flood’s traffic generator targeting the wrong packets-per-second rate, compounded by a fixed post-connect() sleep that let real system-call latency dominate the achieved rate – fixed with a deadline-based scheduler, verified live at 17.58–17.61 packets/sec across six consecutive trials, all six correctly classified (was 3/15); (3) THROTTLE silently defaulting to a TCP-only firewall rule for every attack because the caller populated an empty context, never wiring through the real detected protocol – the exact and sole cause of icmp\_ping\_flood’s 0% figure, confirmed end-to-end before and after the fix; (4) a Suricata rule type (threshold) that never once fires for a UDP flow in our environment, confirmed by testing it in isolation against four real pcaps with a count far below what the traffic contained; (5) our lab-tailored Suricata ruleset covering only 5 of 17 attacks, plus a second, compounding discovery that a partial fix already existed in this project’s own history but had never been deployed to the path Suricata actually reads from; and (6) a capture-buffering bug in which tcpdump’s default behavior only flushes to disk once its buffer fills, losing the final packet of any sufficiently sparse capture if a forced termination is ever needed – fixed with tcpdump’s own -U (packet-buffered) flag.

Every fix was verified with a live, direct repro before being accepted, and the full harness was then re-run under the fixed code (Table II): aggregate detection accuracy reached 99.6%, with 16 of 17 conditions scoring a clean 100%. We report this as the central methodological result of this paper, not a footnote to it: a 16-point accuracy revision is not, on its own, evidence of a well-run study – treating that revision as a set of falsifiable hypotheses, rather than an accepted ceiling, is.

#### B. An adaptive-attacker claim that did not survive scaling

A separate module tests whether identity-based containment (BLOCK\_SOURCE) actually contains a persistent attacker across repeated attempts, in two modes: fixed (the same source every round, a control) and rotate (a fresh, not-yet-blocked source every round). An initial five-round pilot suggested a real finding: a second, fresh spoofed source (.121) reached the sensor and was independently detected and blocked on its first attempt, consistent with BLOCK\_SOURCE’s identity-scoped mechanism (confirmed directly from its implementation) offering no protection against a rotating attacker.

We scaled this to 30 rounds/mode to obtain more than two data points. The scaled run did not produce the

expected volume of clean evidence – instead, reading the raw per-round output revealed a real bug in our own module: rotate-mode’s source selection always chose the first not-yet-blocked address in a fixed pool. Because the second address’s capture came back empty (ambiguous between “blocked” and “not observed,” a distinction our own retry logic is designed to disclose rather than conflate), it stayed first-in-line and was re-selected for all 28 remaining rounds instead of the campaign rotating through the rest of the pool. We fixed the selection logic, added a regression test, expanded the source pool from 6 to 21 addresses, and re-ran (25 rounds/mode, genuinely cycling through 20 distinct spoofed addresses this time, confirmed from the log). Every one of the 20 distinct spoofed addresses produced zero captured packets, on both the initial attempt and the built-in retry (40 real zero-packet observations), in two independent bug-fixed runs. Only the physical attacker’s own real, unspoofed address ever produced capturable traffic.

The honest conclusion this now supports is weaker than the original pilot suggested: the one successful spoofed capture that grounded our original claim could not be reproduced across 40 further attempts. We report this as one unreplicated pilot observation, not confirmed by two larger, bug-fixed follow-up attempts – not as strengthened evidence for source-rotation evasion. The architectural argument (BLOCK\_SOURCE is scoped to the packet’s claimed source address, not the physical sender) still stands independent of this measurement, but we no longer claim repeated Mininet evidence that a rotating attacker gets through. Root-causing why spoofed traffic is not reliably captured in this environment (host-level reverse-path filtering was checked and does not explain it; a per-namespace or virtual-switch-level cause was not further isolated) is left as future work.

#### C. A training-budget fix that looked real and did not generalize

Section V-E reports a reward-function fix that raised PPO’s seed-convergence rate from 8/10 to 9/10, leaving one seed (17) still failing on the identical mistake. Isolating that one seed and retraining it at  $2\times$  and  $4\times$  the deployed timestep budget showed the mismatch move to a different scenario at  $2\times$  and disappear entirely at  $4\times$  – a result that, tested on that one seed alone, looked like a genuine, targeted fix. Generalizing from it was premature: a full ten-seed sweep at the  $4\times$  budget came back to 8/10, not an improvement. Seed 17 did converge, exactly as the isolated test predicted, but seeds 3 and 13 – both fully converged at the original budget – newly failed on two scenarios that had never been a problem for any seed before. Quadrupling the training budget did not fix the aggregate seed-fragility; it moved which seeds and scenarios were most sensitive to initialization, at four times the real training cost. We reverted the change. The deployed recipe and the 9/10 figure in Section V-E are the

reward fix alone, at the original training budget – the best real recipe this investigation found, not the most recent one tried.

## VII. Limitations

Single environment. Every result in this paper comes from one Mininet topology on one resource-constrained (2-CPU,  $\approx 2.9$  GB) virtual machine. No claim here should be read as generalizing to arbitrary IoT hardware, network topologies, or attacker populations; we have not tested beyond this single environment.

No adaptive, threshold-aware attacker. Our robustness measurements (Section V-I) use symmetric, non-adaptive perturbation. A real adversary that probes and adapts to our specific detection thresholds was not modeled.

Shared detection across internal arms. The three policies and two baselines receive identical detection results per trial; our comparison measures response selection and execution, not detection accuracy, which is why the Suricata arm exists as an independent check.

Adaptive-attacker evidence is explicitly weak, as detailed in Section VI-B – an unreplicated pilot observation, not a demonstrated finding.

One condition retains a real, disclosed residual gap. credential\_replay scores 93.3% (14/15); its one miss has a completely empty, unreadable packet capture – a general Mininet/tcpdump capture-flakiness class distinct from the capture-race bug fixed in Section VI-A, and not further resolved in this pass.

## VIII. Ethics and Responsible Disclosure

Every attack technique implemented in this system targets only hosts created inside an isolated Mininet virtual network on a single VM under our own control, with no path to any real external host or network. A systematic self-review of this codebase found and fixed 21 issues, two of which would be real security weaknesses in a production deployment of this code: a multi-source leak in two response handlers, and unsanitized shell interpolation of a source-IP value in two others. Neither reached this paper’s reported evaluation data (verified directly against the harness’s own trial-isolation design and the fact that every source-IP value the harness passes is a registry- or Mininet-derived address, never externally supplied text), and both are documented here alongside what would need fixing before any real deployment, rather than treated as already resolved.

## IX. Conclusion

We presented a registry-driven residential IoT defense system comparing three response-selection policies against naive baselines and an external IDS on real, captured Mininet traffic, with a statistical and ablation treatment – including a formal paired significance test – uncommon in comparable systems papers. Our central positive result is that all three policies substantially and reliably

outperform naive fixed-response and no-response baselines (99.6% verified-response rate versus 29.4% and 5.9%), that policy choice matters specifically at decision boundaries rather than on confidently-classified traffic (McNemar’s exact test,  $p < 1.2 \times 10^{-159}$  against rule-based,  $p = 1.0$  between Stackelberg and PPO), and that this 99.6% figure itself is the product of finding and fixing six real, independently-verified defects, not an assumption. Our central methodological result is that a scale-up is only worth running if it is treated as a test the original finding can fail: a threefold sample-size increase first revealed our headline accuracy had been optimistic, systematic investigation of every regressed condition then recovered and substantially exceeded it, a source-rotation evasion claim did not survive a bug-fixed, higher-powered re-test, and a training-budget fix that looked real on one isolated seed did not generalize to the full population it was meant to help. We report all three corrections in full, including the one that failed after already being committed, rather than retaining whichever number was most favorable at each step. External validity beyond our single controlled environment remains the primary open problem for future work; internal validity, in this paper, was earned through repeated, disclosed self-correction rather than assumed from a single successful run.

## References

- [1] “A realistic experimental comparison of the suricata and snort intrusion-detection systems,” <https://calhoun.nps.edu/server/api/core/bitstreams/6e9ec886-297c-4913-8cc6-80a4c44609a5/content>, naval Postgraduate School.
- [2] “Collaborative device-level botnet detection for internet of things,” <https://www.sciencedirect.com/science/article/pii/S0167404823000822>, scienceDirect.
- [3] “Towards a moving target defense based on stochastic games and honeypots,” <https://www.sciencedirect.com/science/article/pii/S0020025525006206>, scienceDirect.
- [4] “Stackelberg game for resilient collaborative cyber threat detection and response in iot networks,” [https://www.researchgate.net/publication/398764870\\_Stackelberg\\_Game\\_for\\_Resilient\\_Collaborative\\_Cyber\\_Threat\\_Detection\\_and\\_Response\\_in\\_IoT\\_Networks](https://www.researchgate.net/publication/398764870_Stackelberg_Game_for_Resilient_Collaborative_Cyber_Threat_Detection_and_Response_in_IoT_Networks), researchGate.
- [5] “Deep reinforcement learning for intrusion detection in iot: A survey,” <https://arxiv.org/pdf/2405.20038>, arXiv.
- [6] “An optimized reinforcement learning based mtd mutation strategy for securing edge iot against ddos attack,” <https://www.sciencedirect.com/science/article/pii/S2214212625001759>, scienceDirect.